LegalRsearch接口文档

法律研究 (Legal Research) 接口文档

目录

1. 接口概述

核心功能

2. 认证机制

2.1 认证方式

2.2 签名算法

签名原文（Canonical String）

签名示例

2.3 认证失败响应

3. 核心接口

3.1 创建分析会话

接口信息

请求参数

请求示例

query字段增强说明

响应参数

响应示例

错误响应

3.2 法律分析SSE流式接口

接口信息

请求参数

第一轮对话请求示例

后续对话请求示例

SSE响应格式

消息事件 - 增量内容

消息结束事件

错误事件

前端处理示例
错误响应
3.3 获取会话列表
接口信息
请求参数
响应参数
chats数组项结构
响应示例
3.4 获取会话历史
接口信息
请求参数
请求示例
响应参数
chats数组项结构
响应示例
错误响应
3.5 删除会话
接口信息
请求参数
请求示例
响应参数
响应示例
错误响应
3.6 文件上传接口
接口信息
请求参数
支持的文件格式
文件限制
请求示例（cURL）
响应参数
响应示例
调用流程

Python示例
错误响应
3.7 法律分析任务提交接口
接口信息
请求参数
请求示例
响应参数
响应示例
使用流程
错误响应
3.8 法律分析任务状态查询接口
接口信息
请求参数
请求示例
响应参数
响应示例
客户端轮询机制
错误响应
4. SSE流式响应规范
4.1 连接管理
4.2 心跳机制
4.3 事件类型汇总
5. 研究模式说明
5.1 Quick模式（快速分析）
5.2 Search模式（深度研究）
6. 调用流程示例
6.1 完整调用流程（Search模式）
6.2 代码示例（Python）
6.3 代码示例（JavaScript/Node.js）
7. 错误码说明
7.1 响应格式说明
7.2 认证相关错误码

7.3 业务错误码
7.4 错误处理建议
8. 性能与限制
8.1 文件限制
8.2 对话限制
8.3 速率限制
8.4 最佳实践
附录
A. query字段模板
B. 支持的文件格式
C. 输出语言支持
D. 故障排查指南 (FAQ)
E. 技术支持

法律研究 (Legal Research) 接口文档

> **版本**: v1.0
> **更新日期**: 2026-04-23
> **适用对象**: 外部客户集成调用
> **测试环境基础URL**: `https://test.alta-lex.ai`
> **生产环境基础URL**: `https://app.alta-lex.ai`
> **接口路径前缀**: 所有接口路径包含 `/openapi` 前缀

目录

1. 接口概述
2. 认证机制
3. 核心接口
   - 3.1 创建分析会话
   - 3.2 法律分析SSE流式接口
   - 3.3 获取会话列表

    - 3.4 获取会话历史
    - 3.5 删除会话
    - 3.6 法律分析任务提交接口
    - 3.7 法律分析任务结果查询接口
    - 3.8 文件上传接口
4. SSE流式响应规范
5. 研究模式说明
6. 调用流程示例
7. 错误码说明
8. 性能与限制

---

1. 接口概述

法律研究模块提供法律条文检索和深度分析能力，支持两种研究模式：

模式	说明	适用场景	特点
Quick	快速分析模式	简单法律问题、快速查询	响应速度快
Search	深度研究模式	复杂法律研究、深度分析	分析更深入，支持多轮对话（最多20轮）

核心功能

- ✅ 支持自然语言法律问题查询
- ✅ 支持上传参考文件（PDF/DOC/DOCX）
- ✅ 自动识别输出语言
- ✅ SSE流式响应，实时返回分析结果
- ✅ 支持多轮对话交互
- ✅ 会话管理和历史记录查询

---

2. 认证机制

2.1 认证方式

外部客户调用 API 时，必须使用 **Ed25519 请求签名** 进行认证。每个请求须在 HTTP Header 中携带以下字段：

Header	说明	示例
`X-API-Key-ID`	客户生成并已登记的 Key ID	`pk_live_L84_UR4VaR9z8OeqNPVqOq3nbeN2Jf2`
`X-Request-Timestamp`	Unix 秒级时间戳	`1717323152`
`X-Request-Nonce`	16 个十六进制字符（8 字节随机数的 hex）	`9c76d9e45123faaf`
`X-Ed25519-Signature`	对签名原文的 Ed25519 签名，Base64 编码	`X1psD/A1cy7S6qM/UK8liSUepB4L4+2URy+K35n9dc/sTXslr8DaoSp/2Thu43pZzbiwTcXTagRAQLAFa9tgCg==`
`Content-Type`	固定为 `application/json`	`application/json`

请求示例：

```
POST /openapi/createAnalysisSession HTTP/1.1
Host: your-assigned-domain.com
X-API-Key-ID: pk_live_L84_UR4VaR9z8OeqNPVqOq3nbeN2Jf2
X-Request-Timestamp: 1717323152
X-Request-Nonce: 9c76d9e45123faaf
X-Ed25519-Signature: X1psD/A1cy7S6qM/UK8liSUepB4L4+2URy+K35n9dc/sTXslr8DaoSp/2Thu43pZzbiwTcXTagRAQLAFa9tgCg==
Content-Type: application/json

{
  "query": "Analyze the data protection requirements for fintech companies in Hong Kong",
  "researchType": "search"
}
```

2.2 签名算法

签名原文（Canonical String）

按以下顺序用 **ASCII 竖线** `|` 拼接为单行字符串（无首尾空格、无多余分隔符）：

```
{timestamp}|{nonce}|{HTTP方法}|{请求路径}|{查询字符串}|{Body的SHA256十六进制}
```

段	规则
`timestamp`	与 `X-Request-Timestamp` 一致
`nonce`	与 `X-Request-Nonce` 一致
`HTTP方法`	大写：`GET`、`POST` 等
`请求路径`	仅 path，不含 host、不含查询串；例如 `/openapi/createAnalysisSession`
`查询字符串`	原始 query string，无 `?`；无查询参数时为空字符串
`Body的SHA256十六进制`	Body 原始字节（UTF-8）的 SHA-256，输出 64 位小写十六进制；无 Body 时对空字节序列做 SHA-256

空 Body 的 SHA-256（十六进制）：

`e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`

Body 与 JSON：序列化应稳定（键顺序、无多余空格），建议与客户侧实际发送字节一致；常用为 `json.dumps(obj, separators=(',', ':'), ensure_ascii=False)` 再 UTF-8 编码后取哈希。

签名示例

以 `POST /openapi/createAnalysisSession` 为例：

```python
import base64
import hashlib
import json
import secrets
import time
from nacl.signing import SigningKey

# 1. 生成时间戳和 nonce
timestamp = str(int(time.time()))
nonce = secrets.token_hex(8)  # 16 个 hex 字符

# 2. 准备请求体
body = json.dumps({
    "query": "Analyze the data protection requirements for fintech companies in Hong Kong",
    "researchType": "search"
}, separators=(",", ":"), ensure_ascii=False)

# 3. 计算 Body SHA256
body_hash = hashlib.sha256(body.encode("utf-8")).hexdigest()

# 4. 构造签名原文
message = f"{timestamp}|{nonce}|POST|/openapi/createAnalysisSession||{body_hash}"

# 5. Ed25519 签名（PRIVATE_KEY_BASE64 为客户本地生成的私钥）
private_key = SigningKey(base64.b64decode(PRIVATE_KEY_BASE64))
signed = private_key.sign(message.encode("utf-8"))
signature = base64.b64encode(bytes(signed)[:64]).decode("ascii")

# 6. 构建请求头
headers = {
    "X-API-Key-ID": API_KEY_ID,
    "X-Request-Timestamp": timestamp,
    "X-Request-Nonce": nonce,
    "X-Ed25519-Signature": signature,
    "Content-Type": "application/json",
}
```

> **密钥生成与登记**：客户需在本地生成 Ed25519 密钥对，仅将公钥提交 AltaLex 登记。详细流程请参考《AltaLex API 接入技术规范》。

2.3 认证失败响应

```json
{
  "status": "error",
  "error": {
    "code": "A01003",
    "message": "Invalid or inactive API key",
    "timestamp": "2026-04-20T10:30:00+08:00",
    "path": "/openapi/createAnalysisSession"
  },
  "traceId": "1706355600000-a1b2c3d4"
}
```

---

3. 核心接口

3.1 创建分析会话

创建法律研究会话，保存研究参数并返回sessionId。

接口信息

- **接口路径**: `/openapi/createAnalysisSession`
- **请求方法**: POST
- **Content-Type**: application/json

请求参数

参数名	类型	必填	默认值	说明
query	string	**是**	–	研究问题，可包含Practice Area、Jurisdiction、Output Language等字段

researchType	string	**否**	`"search"`	研究模式：`"quick"`=快速分析，`"search"`=深度研究。**不传时默认为"search"**
fileUrls	array[string]	**否**	`null`（不传）	参考文件URL列表，最多10个文件。**不传时表示无参考文件**

请求示例

```
{
  "query": "Analyze the data protection requirements for fintech companies in Hong Kong under PDPO and relevant regulations",
  "researchType": "search",
  "fileUrls": [
    "https://storage.example.com/documents/pdpo_guidelines.pdf",
    "https://storage.example.com/documents/fintech_regulations.docx"
  ]
}
```

query字段增强说明

query支持结构化字段，系统会自动解析：

```
Practice Area: Data Protection & Privacy
Jurisdiction: Hong Kong
Output Language: English
Background: Our client is a fintech company providing digital payment services in Hong Kong
Query: What are the key compliance requirements under PDPO for handling customer data?
```

Output Language说明：

- 如果未指定，系统会自动识别query内容并推荐输出语言
- 支持格式：`Output Language: 中文` 或 `Output Language: English`

- 自动识别结果会存储在session中，后续对话自动应用
- **自动识别触发条件**：当query中未包含Output Language字段时触发
- **识别失败处理**：默认使用English作为输出语言

响应参数

参数名	类型	说明
status	string	操作状态："success"
sessionId	string	会话ID，用于后续接口调用
message	string	提示信息
traceId	string	请求追踪ID

响应示例

```
{
  "status": "success",
  "message": "Session created successfully",
  "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c",
  "traceId": "1706355600000-a1b2c3d4"
}
```

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A02002	File count exceeds limit. Maximum 10 files allowed.	文件数量超限
A02002	Invalid researchType: xxx	研究类型参数错误

3.2 法律分析SSE流式接口

执行法律分析，通过SSE流式返回分析结果。

接口信息

- **接口路径**: `/openapi/legalAnalysisSse`
- **请求方法**: POST
- **Content-Type**: application/json
- **Accept**: text/event-stream

请求参数

第一轮对话与后续对话的区别：

对话轮次	sessionId	query	researchType	说明
第一轮	✅ **必填**	❌ **不传**	❌ **不传**	query和researchType从创建会话时保存的数据中自动获取
后续对话	✅ **必填**	✅ **必填**	⚠️ **建议传入**	必须提供query；researchType建议与创建时保持一致

重要提示：

- **第一轮对话**：只需传 `sessionId` ，系统自动使用创建会话时的query、fileUrls和researchType
- **后续对话**：必须传 `sessionId` 和 `query` ；可选择性传入fileUrls和researchType
- **researchType获取规则**：
  - 第一轮：从session中获取（创建会话时保存的值）
  - 后续：从请求参数获取；**如不传则默认为"search"**
  - **强烈建议**：后续对话的researchType应与创建会话时保持一致，否则可能导致上下文关联问题

参数名	类型	必填	默认值	说明
sessionId	string	**是**	–	会话ID（从创建会话接口获取）

chatId	string	**否**	自动生成	对话ID，不提供则系统自动生成
query	string	**条件必填**	–	第一轮对话**不传**（从session获取）；后续对话**必填**
fileUrls	array[string]	**否**	`null`（不传）	参考文件URL列表，最多10个。**不传时表示不使用文件**
researchType	string	**否**	`"search"`	研究模式。第一轮从session获取；后续从请求参数获取，**不传时默认为"search"。不建议切换模式**

第一轮对话请求示例

第一轮对话时，query和fileUrls从session中获取，请求体中只需传sessionId：

```
{
  "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c"
}
```

后续对话请求示例

后续对话必须提供query参数：

```
{
  "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c",
  "chatId": "chat_1a2b3c4d5e6f",
  "query": "What are the penalties for non-compliance with PDPO?",
  "researchType": "search"
}
```

SSE响应格式

采用Server-Sent Events (SSE)流式响应，实时返回分析内容。

消息事件 – 增量内容

每个消息事件包含一个文本片段（增量输出）：

```
data: {
  "message": "Based on the analysis of Hong Kong's Personal Data (Privacy) Ordinance (PDPO), ",
  "is_finished": false
}
```

```
data: {
  "message": "fintech companies must comply with the following key requirements:",
  "is_finished": false
}
```

消息结束事件

表示消息生成结束：

```
data: {
  "message": "",
  "is_finished": true,
  "conversation_id": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c"
}
```

错误事件

```
data: {
  "error": "Error occurred while processing request: API timeout",
  "message": "",
  "is_finished": true
}
```

前端处理示例

```javascript
const eventSource = new EventSource('/openapi/legalAnalysisSse');

let fullAnswer = '';

eventSource.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.error) {
    // 显示错误信息
    showError(data.error);
    eventSource.close();
    return;
  }

  if (data.is_finished) {
    // 显示完成
    eventSource.close();
    return;
  }

  // 累积显示增量内容
  fullAnswer += data.message;
  displayContent(fullAnswer);
};

eventSource.onerror = (error) => {
  console.error('SSE Error:', error);
  eventSource.close();
};
```

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A02002	Query parameter not found in session	第一轮对话但session中无query
A02002	Subsequent conversations must include query parameter	后续对话未提供query

A02002	File count exceeds limit. Maximum 10 files allowed.	文件数量超限
A02002	Invalid researchType: xxx	研究类型参数错误
A03002	Session not found or access denied	会话不存在或无权限

3.3 获取会话列表

获取当前用户的所有法律研究会话列表。

接口信息

- **接口路径**: `/openapi/getAnalysisSessionList`
- **请求方法**: GET

请求参数

无需请求参数，通过认证信息获取当前用户的会话列表。

响应参数

参数名	类型	说明
status	string	操作状态："success"
message	string	提示信息
chats	array	会话列表
traceId	string	请求追踪ID

chats数组项结构

参数名	类型	说明
sessionId	string	会话ID

sessionName	string	会话名称
title	string	会话标题（第一次查询内容）

响应示例

```
{
  "status": "success",
  "message": "",
  "chats": [
    {
      "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c",
      "sessionName": "Legal Research Session",
      "title": "Analyze the data protection requirements for fintech companies in Hong Kong"
    },
    {
      "sessionId": "sess_lr_1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d",
      "sessionName": "Legal Research Session",
      "title": "What are the employment law requirements in Singapore?"
    }
  ],
  "traceId": "1706355600000-a1b2c3d4"
}
```

3.4 获取会话历史

获取指定会话的完整对话历史。

接口信息

- **接口路径**: `/openapi/getAnalysisSessionHistory`
- **请求方法**: GET

请求参数

参数名	类型	必填	说明

sessionId	string	是	会话ID（通过query参数传递）

请求示例

```
GET /openapi/getAnalysisSessionHistory?sessionId=sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c
```

响应参数

参数名	类型	说明
status	string	操作状态："success"
message	string	提示信息
chats	array	对话历史列表
researchType	string	研究模式："quick"或"search"，表示当前会话使用的研究模式
traceId	string	请求追踪ID

chats数组项结构

参数名	类型	说明
chatId	string	对话ID
query	string	用户提问
answer	string	系统回答（处理中或失败时可能为空或包含错误信息）
status	string	**对话状态**，详见下方状态值说明

status字段状态值说明：

状态值	含义	说明	answer字段表现

`processing`	处理中	任务正在执行，尚未完成	通常为空字符串 `""`
`completed`	已完成	任务成功完成	包含完整的分析结果
`error`	错误	任务执行失败	包含错误信息描述

状态转换流程：

```
创建对话 → processing（处理中） → completed（完成）
                                      ↓
                                  error（错误）
```

使用建议：

- 当 `status="processing"` 时，表示任务仍在后台执行，可稍后再次查询
- 当 `status="completed"` 时， `answer` 字段包含完整结果
- 当 `status="error"` 时， `answer` 字段包含错误信息，可根据错误内容决定是否重试

响应示例

```json
{
  "status": "success",
  "message": "",
  "chats": [
    {
      "chatId": "chat_1a2b3c4d5e6f",
      "query": "Analyze the data protection requirements for fintech companies in Hong Kong",
      "answer": "Based on the analysis of Hong Kong's Personal Data (Privacy) Ordinance (PDPO)...",
      "status": "completed"
    },
    {
      "chatId": "chat_2b3c4d5e6f7a",
      "query": "What are the penalties for non-compliance?",
      "answer": "",
      "status": "processing"
    },
    {
      "chatId": "chat_3c4d5e6f7a8b",
      "query": "Compare with Singapore PDPA",
      "answer": "Error occurred while processing request",
      "status": "error"
    }
  ],
  "researchType": "search",
  "traceId": "1706355600000-a1b2c3d4"
}
```

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A02002	Missing parameters	缺少sessionId参数
A03002	Session not found or no access	会话不存在或无权限

3.5 删除会话

删除指定的法律研究会话及其所有对话记录。

接口信息

- **接口路径**: `/openapi/removeAnalysisSession`
- **请求方法**: POST
- **Content-Type**: application/json

请求参数

参数名	类型	必填	说明
sessionId	string	是	要删除的会话ID

请求示例

```
1 {
2   "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c"
3 }
```

响应参数

参数名	类型	说明
status	string	操作状态："success"
sessionId	string	已删除的会话ID
message	string	提示信息
traceId	string	请求追踪ID

响应示例

```
{
  "status": "success",
  "message": "Session deleted successfully",
  "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c",
  "traceId": "1706355600000-a1b2c3d4"
}
```

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A03002	Session not found	会话不存在
B00001	Failed to delete session	删除会话失败

3.6 文件上传接口

上传文件到服务器，获得文件存储路径，用于法律分析接口的fileUrls参数。

接口信息

- **接口路径**: `/openapi/uploadFile`
- **请求方法**: POST
- **Content-Type**: multipart/form-data

请求参数

参数名	类型	必填	说明
file	file	是	要上传的文件

business_type	string	否	业务类型，用于控制支持的文件格式。传 `"2"` 表示法律研究模式（额外支持图片格式），不传则仅支持PDF和Word格式

支持的文件格式

基础格式（所有模式）：

- PDF (.pdf)
- Word (.doc, .docx)

法律研究模式额外支持（business_type=`"2"`）：

- 图片 (.jpg, .jpeg, .png)

文件限制

限制项	数值
文件大小	最大10MB
文件数量	单次分析最多10个文件

请求示例（cURL）

```
curl -X POST "https://your-assigned-domain.com/openapi/uploadFile" \
  -H "X-API-Key-ID: pk_live_xxx" \
  -H "X-Request-Timestamp: 1717323152" \
  -H "X-Request-Nonce: 9c76d9e45123faaf" \
  -H "X-Ed25519-Signature: xxx" \
  -F "file=@/path/to/legal_document.pdf" \
  -F "business_type=2"
```

响应参数

参数名	类型	说明

status	string	操作状态："success"
message	string	提示信息
task_data	string	文件存储路径，用于fileUrls参数
traceId	string	请求追踪ID

响应示例

上传成功：

```
{
  "status": "success",
  "message": "File uploaded successfully",
  "task_data": "uploads/20260423/legal_doc_abc123.pdf",
  "traceId": "1706355600000-a1b2c3d4"
}
```

文件已存在（幂等响应）：

```
{
  "status": "success",
  "message": "File already exists",
  "task_data": "uploads/20260423/legal_doc_abc123.pdf",
  "traceId": "1706355600000-a1b2c3d4"
}
```

调用流程

```
┌──────────────────────────────────────────────────────────┐
│  Step 1: 上传文件                                          │
│  POST /openapi/uploadFile                                  │
│  → 返回 task_data (文件存储路径)                            │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Step 2: 创建会话或进行分析                                  │
│  POST /openapi/createAnalysisSession                       │
│  或 POST /openapi/legalAnalysisSse                         │
│  → 将 task_data 作为 fileUrls 参数传入                      │
│  → 示例: {"fileUrls": ["uploads/20260423/legal_doc_abc123.pdf"]}│
└──────────────────────────────────────────────────────────┘
```

Python示例

```python
import httpx

# Step 1: 上传文件
with open("legal_document.pdf", "rb") as f:
    files = {"file": ("legal_document.pdf", f, "application/pdf")}
    data = {"business_type": "2"}

    response = httpx.post(
        "https://your-assigned-domain.com/openapi/uploadFile",
        headers=signed_headers,  # Ed25519签名头
        files=files,
        data=data
    )

    file_path = response.json()["task_data"]
    print(f"文件上传成功: {file_path}")

# Step 2: 使用文件路径进行分析
analysis_request = {
    "query": "Analyze this legal document",
    "fileUrls": [file_path]  # 使用上传返回的路径
}
```

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A02002	No file selected	未选择文件
A02002	Invalid file type	不支持的文件类型
A02002	File size exceeds limit	文件大小超过10MB限制
B00001	File upload failed	文件上传失败

3.7 法律分析任务提交接口

提交法律分析任务（非SSE异步模式），立即返回任务ID，后台异步执行。

接口信息

- **接口路径**: `/openapi/legalAnalysisTask`
- **请求方法**: POST
- **Content-Type**: application/json

请求参数

与 3.2 法律分析SSE流式接口 的请求参数完全相同。

参数名	类型	必填	默认值	说明
sessionId	string	**是**	–	会话ID（从创建会话接口获取）
chatId	string	**否**	自动生成	对话ID，不提供则系统自动生成
query	string	**条件必填**	–	第一轮对话**不传**（从session获取）；后续对话**必填**

fileUrls	array[string]	**否**	`null` （不传）	参考文件URL列表，最多10个。**不传时表示不使用文件**
researchType	string	**否**	`"search"`	研究模式。第一轮从session获取；后续从请求参数获取，**不传时默认为"search"。不建议切换模式**

请求示例

第一轮对话：

```
{
  "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c"
}
```

后续对话：

```
{
  "sessionId": "sess_lr_8f3a9b2c1d4e5f6a7b8c9d0e1f2a3b4c",
  "chatId": "chat_1a2b3c4d5e6f",
  "query": "What are the penalties for non-compliance with PDPO?",
  "researchType": "search"
}
```

响应参数

参数名	类型	说明
status	string	操作状态："success"
chatId	string	聊天ID，用于后续查询任务状态和结果

taskStatus	string	任务状态："processing"=处理中，"completed"=已完成，"error"=错误
message	string	提示信息
traceId	string	请求追踪ID

响应示例

任务提交成功：

```
{
  "status": "success",
  "message": "Task submitted successfully",
  "chatId": "chat_1a2b3c4d5e6f",
  "taskStatus": "processing",
  "traceId": "1706355600000-a1b2c3d4"
}
```

任务已完成（幂等响应）：

```
{
  "status": "success",
  "message": "Task already completed",
  "chatId": "chat_1a2b3c4d5e6f",
  "taskStatus": "completed",
  "traceId": "1706355600000-a1b2c3d4"
}
```

使用流程

1. 调用本接口提交任务，立即获得 `chatId`
2. 通过 3.7 法律分析任务结果查询接口 查询任务状态和结果
3. 或通过 3.4 获取会话历史 接口查询完整对话历史

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A02002	Query parameter not found in session	第一轮对话但session中无query
A02002	Subsequent conversations must include query parameter	后续对话未提供query
A02002	File count exceeds limit. Maximum 10 files allowed.	文件数量超限
A02002	Invalid researchType: xxx	研究类型参数错误
A03002	Session not found or access denied	会话不存在或无权限

3.8 法律分析任务状态查询接口

查询法律分析任务的执行状态和结果（非阻塞式，立即返回）。

接口信息

- **接口路径**: `/openapi/getLegalAnalysisTaskResult`
- **请求方法**: GET

请求参数

参数名	类型	必填	说明
chatId	string	是	聊天ID（通过query参数传递）

请求示例

```
GET /openapi/getLegalAnalysisTaskResult?chatId=chat_1a2b3c4d5e6f
```

响应参数

参数名	类型	说明
status	string	操作状态："success"
chatId	string	聊天ID
query	string	用户查询内容
answer	string	任务结果（完成时返回完整结果，失败时返回错误信息）
taskStatus	string	**任务状态**，详见下方状态值说明
thinking	string	思考过程（Search模式，可选）
sources	array	引用来源（Search模式，可选）
traceId	string	请求追踪ID

taskStatus字段状态值说明：

状态值	含义	说明	answer字段表现	客户端操作
`processing`	处理中	任务正在后台执行	通常为空字符串 `""`	继续轮询
`completed`	已完成	任务成功完成	包含完整的分析结果	停止轮询，处理结果
`error`	错误	任务执行失败	包含错误信息描述	停止轮询，根据错误决定是否重试

响应示例

任务进行中：

```
{
  "status": "success",
  "chatId": "chat_1a2b3c4d5e6f",
  "query": "Analyze the data protection requirements for fintech companies in Hong Kong",
  "answer": "",
  "taskStatus": "processing",
  "thinking": null,
  "sources": null,
  "traceId": "1706355600000-a1b2c3d4"
}
```

任务已完成：

```
{
  "status": "success",
  "chatId": "chat_1a2b3c4d5e6f",
  "query": "Analyze the data protection requirements for fintech companies in Hong Kong",
  "answer": "Based on the analysis of Hong Kong's Personal Data (Privacy) Ordinance (PDPO)...",
  "taskStatus": "completed",
  "thinking": "First, I will analyze the key provisions of PDPO...",
  "sources": [
    {
      "title": "PDPO Official Guidelines",
      "url": "https://example.com/pdpo"
    }
  ],
  "traceId": "1706355600000-a1b2c3d4"
}
```

任务失败：

```
{
  "status": "success",
  "chatId": "chat_1a2b3c4d5e6f",
  "query": "Analyze the data protection requirements",
  "answer": "Error occurred while processing request: API timeout",
  "taskStatus": "error",
  "thinking": null,
  "sources": null,
  "traceId": "1706355600000-a1b2c3d4"
}
```

客户端轮询机制

本接口立即返回当前任务状态。便于客户端需自行实现轮询逻辑：

轮询建议：

- **检查间隔**: 5–10秒（避免过频繁请求）
- **最大等待时间**: 30分钟（Quick模式约30秒，Search模式约8–12分钟）
- **超时处理**: 超时后可继续轮询或放弃
- **错误处理**: 网络错误时重试2–3次

错误响应

注意：所有错误响应的HTTP状态码均为200，通过响应体中的 `status` 和 `error.code` 字段区分错误类型。

错误码	错误信息	说明
A02002	Missing chatId parameter	缺少chatId参数
A03003	Chat not found or access denied	聊天不存在或无权限

---

4. SSE流式响应规范

4.1 连接管理

- SSE连接建立后，服务器会持续推送事件
- 客户端应监听 `is_finished=true` 或 `error` 字段来关闭连接
- 建议设置连接超时时间（推荐15分钟）

4.2 心跳机制

服务器会定期发送心跳事件保持连接活跃，客户端应忽略心跳事件：

```
: heartbeat 1

```

4.3 事件类型汇总

事件类型	说明	数据字段
消息增量	增量内容输出	message, is_finished
消息结束	消息生成结束	message(""), is_finished(true), conversation_id
错误	错误事件	error, message(""), is_finished(true)
心跳	心跳保持	以冒号开头的注释行

---

5. 研究模式说明

5.1 Quick模式（快速分析）

特点：

- 响应速度较快
- 适合简单法律问题查询
- 支持文件上传

使用场景：

- 基础法律概念查询
- 简单法规解读
- 快速法律意见

5.2 Search模式（深度研究）

特点：

- 分析更深入全面
- 支持多轮对话（最多20轮）
- 支持文件上传
- 自动保存第一轮报告用于后续对话上下文

使用场景：

- 复杂法律问题分析
- 多法域比较研究
- 深度合规分析
- 需要引用多个法律来源的研究

多轮对话限制：

- Quick模式最多支持20轮对话
- Search模式最多支持20轮对话
- 超过限制后会返回错误提示
- 需要创建新会话继续对话
- **上下文参考机制**：会话中的每一轮对话（包括用户的query和系统的answer）都会作为后续回答的上下文参考，确保对话连贯性

---

6. 调用流程示例

6.1 完整调用流程（Search模式）

```
┌──────────────────────────────────────────────────────────┐
│  Step 1: 创建分析会话                                      │
│  POST /openapi/createAnalysisSession                      │
│  → 返回 sessionId                                         │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Step 2: 执行法律分析（SSE流式）                              │
│  POST /openapi/legalAnalysisSse                           │
│  → SSE流式返回分析结果                                       │
│  → 自动保存首轮报告用于后续对话                               │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Step 3: 后续对话（可选）                                    │
│  POST /openapi/legalAnalysisSse（带新query）                 │
│  → 基于首轮报告和对话历史继续分析                             │
│  → 最多20轮对话                                              │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Step 4: 查询历史记录（可选）                                 │
│  GET /openapi/getAnalysisSessionHistory                   │
│  → 获取完整对话历史                                          │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  Step 5: 删除会话（可选）                                    │
│  POST /openapi/removeAnalysisSession                      │
│  → 清理会话数据                                              │
└──────────────────────────────────────────────────────────┘
```

6.2 代码示例（Python）

```python
import base64
import hashlib
import json
import secrets
import time

import httpx
from nacl.signing import SigningKey

# 请替换为实际分配的域名
BASE_URL = "https://your-assigned-domain.com/openapi"
API_KEY_ID = "pk_live_your_key_id_here"
PRIVATE_KEY_BASE64 = "your_base64_encoded_private_key"


def generate_signed_headers(method: str, path: str, body: str = "") -> dict:
    """生成带 Ed25519 签名的请求头"""
    timestamp = str(int(time.time()))
    nonce = secrets.token_hex(8)  # 16 个 hex 字符
    body_hash = hashlib.sha256(body.encode("utf-8")).hexdigest()
    message = f"{timestamp}|{nonce}|{method}|{path}||{body_hash}"

    private_key = SigningKey(base64.b64decode(PRIVATE_KEY_BASE64))
    signed = private_key.sign(message.encode("utf-8"))
    signature = base64.b64encode(bytes(signed)[:64]).decode("ascii")

    return {
        "X-API-Key-ID": API_KEY_ID,
        "X-Request-Timestamp": timestamp,
        "X-Request-Nonce": nonce,
        "X-Ed25519-Signature": signature,
        "Content-Type": "application/json",
    }


# Step 1: 创建会话
body = json.dumps({
    "query": "What are the key data protection requirements under Hong Kong PDPO?",
    "researchType": "search"
}, separators=(",", ":"), ensure_ascii=False)

headers = generate_signed_headers("POST", "/openapi/createAnalysisSession", body)

```

```python
create_response = httpx.post(f"{BASE_URL}/createAnalysisSession", headers=headers, content=body)

session_data = create_response.json()
session_id = session_data["sessionId"]
print(f"Session created: {session_id}")


# Step 2: 执行分析（SSE流式）
def run_analysis():
    body = json.dumps({"sessionId": session_id}, separators=(",", ":"), ensure_ascii=False)
    headers = generate_signed_headers("POST", "/openapi/legalAnalysisSse", body)
    headers["Accept"] = "text/event-stream"

    with httpx.stream("POST", f"{BASE_URL}/legalAnalysisSse", headers=headers, content=body) as response:
        full_answer = ""
        for line in response.iter_lines():
            if line.startswith("data: "):
                data = json.loads(line[6:])
                if data.get("error"):
                    print(f"\n\nError: {data['error']}")
                    break
                if data.get("is_finished"):
                    print("\n\nAnalysis complete!")
                    break
                full_answer += data.get("message", "")
                print(data.get("message", ""), end="", flush=True)


# Step 3: 获取历史
headers = generate_signed_headers("GET", "/openapi/getAnalysisSessionHistory")
history_response = httpx.get(
    f"{BASE_URL}/getAnalysisSessionHistory",
    headers=headers,
    params={"sessionId": session_id}
)
```

6.3 代码示例（JavaScript/Node.js）

```javascript
const crypto = require('crypto');
const axios = require('axios');
const nacl = require('tweetnacl');

// 请替换为实际分配的域名
const BASE_URL = 'https://your-assigned-domain.com/openapi';
const API_KEY_ID = process.env.ALTA_KEY_ID;
const PRIVATE_KEY_BASE64 = process.env.ALTA_PRIVATE_KEY;

function generateSignedHeaders(method, path, bodyObj = null) {
  const timestamp = Math.floor(Date.now() / 1000).toString();
  const nonce = crypto.randomBytes(8).toString('hex');
  const bodyStr = bodyObj ? JSON.stringify(bodyObj) : '';
  const bodyHash = crypto.createHash('sha256').update(bodyStr).digest('hex');
  const message = `${timestamp}|${nonce}|${method}|${path}||${bodyHash}`;

  const secretKey = Buffer.from(PRIVATE_KEY_BASE64, 'base64');
  const sig = nacl.sign.detached(Buffer.from(message, 'utf8'), secretKey);
  const sigB64 = Buffer.from(sig).toString('base64');

  return {
    'X-API-Key-ID': API_KEY_ID,
    'X-Request-Timestamp': timestamp,
    'X-Request-Nonce': nonce,
    'X-Ed25519-Signature': sigB64,
    'Content-Type': 'application/json',
  };
}

async function createSession() {
  const body = {
    query: 'What are the key data protection requirements under Hong Kong PDPO?',
    researchType: 'search'
  };
  const headers = generateSignedHeaders('POST', '/openapi/createAnalysisSession', body);
  const response = await axios.post(`${BASE_URL}/createAnalysisSession`, body, { headers });
  return response.data.sessionId;
}

async function runAnalysis(sessionId) {
  // 注意：SSE 流式接口需要使用支持流式的 HTTP 客户端
  // 以下为概念示例，实际请使用 eventsource 或 fetch 的 ReadableStream
```

```
  const body = { sessionId };
  const headers = generateSignedHeaders('POST', '/openapi/legalAnalysisSse', body);
  headers['Accept'] = 'text/event-stream';

  // 使用 EventSource 或 fetch 处理 SSE 流
  // 此处为概念示例，具体实现请参考前端处理示例
  console.log('SSE stream processing...');
}

// 使用示例
(async () => {
  const sessionId = await createSession();
  console.log(`Session created: ${sessionId}`);
  await runAnalysis(sessionId);
})();
```

---

7. 错误码说明

7.1 响应格式说明

重要：本项目所有API的HTTP状态码统一为200，通过响应体中的 `status` 字段区分成功或失败：

成功响应：

```
{
  "status": "success",
  "message": "...",
  "traceId": "1706355600000-a1b2c3d4"
}
```

错误响应：

```json
{
  "status": "error",
  "error": {
    "code": "A03001",
    "message": "Session not found",
    "details": null,
    "timestamp": "2026-04-20T10:30:00+08:00",
    "path": "/openapi/legalAnalysisSse"
  },
  "traceId": "1706355600000-a1b2c3d4"
}
```

客户端应通过以下方式判断请求结果：

1. 检查HTTP状态码是否为200（网络层成功）
2. 检查响应体中的 `status` 字段："success"或"error"
3. 如果 `status` 为"error"，通过 `error.code` 获取业务错误码

7.2 认证相关错误码

错误码	错误信息	说明	解决方案
A01003	Invalid or inactive API key	API Key 无效或未激活	检查 Key ID 是否正确，确认是否已完成公钥登记
A01004	Request timestamp is outside the allowed window	请求时间戳过期（超出 ±5 分钟窗口）	校准客户端服务器时间
A01005	Signature verification failed	签名验证失败	检查签名原文构造、私钥格式、Body 哈希计算
A01008	Duplicate request nonce (replay detected)	请求重放（Nonce 重复）	每个请求使用唯一的 Nonce
A01009	Client IP is not allowed for this API key	客户端 IP 不在白名单	联系 AltaLex 更新 IP 白名单

A01010	Invalid parameters	验签参数格式错误	检查请求头是否完整
A02001	Missing required API signature headers	缺少必要的签名请求头	确保携带所有必需的签名Header

7.3 业务错误码

错误码	错误信息	说明	解决方案
A02002	Required parameters are missing	缺少必填参数	检查请求参数是否完整
A03001	Session not found	会话不存在	检查sessionId是否正确
A03002	Session not found or no access	会话不存在或无权限	确认会话归属和认证信息
A02002	File count exceeds limit	文件数量超限	文件数量不超过10个
A02002	Invalid researchType	研究类型参数错误	使用"quick"或"search"
A05001	The conversation has exceeded the maximum limit of 20 turns	超过对话轮次限制	创建新会话继续对话

7.4 错误处理建议

错误处理最佳实践：

```python
import httpx

response = httpx.post(url, headers=headers, json=data)

# 步骤1: 检查HTTP状态码（网络层）
if response.status_code != 200:
    print(f"网络错误: HTTP {response.status_code}")
    # 处理网络层错误（如DNS解析失败、连接超时等）

# 步骤2: 解析响应体
result = response.json()

# 步骤3: 检查业务状态
if result.get("status") == "error":
    error_code = result["error"]["code"]
    error_message = result["error"]["message"]
    trace_id = result.get("traceId")

    print(f"业务错误: {error_code} - {error_message}")
    print(f"TraceID: {trace_id}")  # 联系技术支持时提供

    # 根据错误码处理
    if error_code == "A03002":
        # Session不存在，需要重新创建
        create_new_session()
    elif error_code == "A01004":
        # 时间戳过期，重新生成签名
        retry_with_new_timestamp()
    elif error_code.startswith("A04"):
        # 账户相关问题（余额不足等）
        notify_user_to_recharge()
else:
    # 请求成功
    print("请求成功")
    process_success_response(result)
```

错误重试策略：

错误码类型	建议处理	重试次数
A01xxx（认证错误）	检查签名配置，不建议重试	0
A02xxx（参数错误）	修正参数后重试	1–2
A03xxx（资源不存在）	创建新资源后重试	1

A04xxx（账户问题）	通知用户，不重试	0
B00xxx（系统错误）	稍后重试	2–3
C07xxx（第三方错误）	稍后重试	2–3

重要提示：

- 记录 `traceId` 便于问题追踪和技术支持
- 重试时建议使用递增间隔（1秒、2秒、4秒）
- 对于认证错误（A01xxx），应先修正配置再重试

---

8. 性能与限制

8.1 文件限制

限制项	数值	说明
最大文件数量	10个	单次请求最多上传10个文件
支持文件格式	PDF, DOC, DOCX	DOC/DOCX会自动转换为PDF
文件大小限制	10MB	单个文件最大不超过10MB

8.2 对话限制

限制项	Quick模式	Search模式
最大对话轮次	20轮	20轮
响应时间	平均约30秒（视问题复杂度而定）	平均约8–12分钟（视研究深度而定）
文件支持	✅	✅
多轮对话	✅	✅

8.3 速率限制

	限制项	数值
测试环境	并发请求数	单用户不超过5个并发（时间窗口：1分钟）
	每日请求上限	根据合同协议确定
	超限处理	返回业务错误码A02002（请求过于频繁），需在1分钟后重试
生产环境	生产环境需求和限制请再发布上线前进行协商沟通	

注意：如需更高并发限制，请联系AltaLex支持团队进行协商定制。

8.4 最佳实践

1. **文件处理**
   - 优先使用PDF格式，避免转换延迟
   - 使用 `uploadFile` 接口上传文件获取 `task_data` 路径
   - 将 `task_data` 作为 `fileUrls` 参数传入分析接口
   - 确保文件格式为PDF/DOC/DOCX，大小不超过10MB
   - 上传前验证文件完整性
2. **会话管理**
   - 及时清理不需要的会话
   - 合理使用多轮对话，避免频繁创建新会话
   - 保存sessionId用于后续查询
3. **错误处理**
   - 实现重试机制处理网络错误（建议重试2-3次，间隔递增）
   - 监听SSE的error事件
   - 记录traceId便于问题追踪
   - **幂等性保障**：重复提交相同请求不会创建重复记录
4. **性能优化**

- 使用连接池管理 HTTP 连接
- 合理设置超时时间（推荐15分钟）

附录

A. query字段模板

```
Practice Area: [法律领域，如 Data Protection, Employment Law, Corporate Governance]
Jurisdiction: [法域，如 Hong Kong, Singapore, PRC]
Output Language: [输出语言，如 English, 中文]
Background: [背景信息，可选]
Query: [具体问题]
```

B. 支持的文件格式

格式	扩展名
PDF	.pdf
Word	.doc
Word	.docx

文件调用逻辑：

法律研究接口支持通过上传接口使用文件：

使用上传接口（推荐）

1. **上传文件**：调用 `POST /openapi/uploadFile` 接口上传文件
2. **获取路径**：从响应中获得 `task_data` 字段（文件存储路径）
3. **传递路径**：将 `task_data` 作为 `fileUrls` 参数传入分析接口
4. **系统处理**：
   - 系统会读取上传的文件
   - 解析文件内容用于法律分析

重要说明：

- 所有文件必须通过上传接口上传后使用，不支持直接传入外部文件链接
- 上传成功后返回的 `task_data` 路径有效期与系统存储策略一致
- 如需使用已上传的文件，可在多个会话中复用相同的 `task_data` 路径
- 推荐使用上传接口，可以获得更好的文件管理和安全性

C. 输出语言支持

系统支持多种输出语言，包括但不限于：

- English
- 中文（简体/繁体）
- 日本語
- 한국어
- Español
- Français
- Deutsch
- ...

如果未指定Output Language，系统会自动识别query内容并推荐合适的语言。

D. 故障排查指南 (FAQ)

Q1: SSE连接超时怎么办?

- A: 建议设置连接超时时间为15分钟。如果超时，检查网络连接并重试。Search模式可能需要8–12分钟。

Q2: 如何判断任务是否完成?

- A:
  - SSE模式：监听 `is_finished=true` 事件
  - Task模式：调用 `getLegalAnalysisTaskResult` 接口，检查 `taskStatus` 字段
    - `processing`：任务进行中，继续轮询
    - `completed`：任务完成，answer字段包含结果
    - `error`：任务失败，answer字段包含错误信息
  - 或调用 `getAnalysisSessionHistory` 接口查看会话历史

Q3: 文件上传失败如何处理?

- A: 确保通过 `uploadFile` 接口上传文件，文件格式为PDF/DOC/DOCX，大小不超过10MB。上传成功后使用返回的 `task_data` 路径作为 `fileUrls` 参数。

Q4: 为什么返回"Query parameter not found in session"错误?

- A: 第一轮对话时，系统会从创建会话时保存的query中获取。请检查 `createAnalysisSession` 接口是否正确传递了query参数。

Q5: 如何切换研究模式?

- A: **一个会话只能使用一种研究模式，不支持切换。** 研究模式由 `createAnalysisSession` 接口的 `researchType` 参数决定（默认为"search"）。后续对话建议保持与创建时相同的 `researchType` ，否则可能导致对话上下文无法正确关联。如需切换模式，请创建新的会话。

Q6: 响应时间过长如何处理?

- A:
  - Quick模式平均约30秒，如超过2分钟建议重试
  - Search模式平均约8–12分钟，请耐心等待或使用Task模式异步查询
  - 可通过 `getAnalysisSessionHistory` 接口查看任务状态

Q7: 如何实现请求重试?

- A:
  - 网络错误：重试2–3次，间隔递增（1秒、2秒、4秒）
  - 限流错误（A02002）：等待1分钟后重试
  - Task轮询：
    - 推荐轮询间隔：2–5秒
    - Quick模式：约30秒后完成
    - Search模式：约8–12分钟完成
    - 超时后（30分钟）可继续轮询或放弃
  - **重要**：重试时使用相同的chatId可保证幂等性

E. 技术支持

如有问题或需要技术支持，请联系Alta Lex支持人员

- 提供traceId/sessionId/chatId以便快速定位问题

文档版本历史

版本	日期	说明
v1.0	2026-04-23	外部客户集成调用